VIDYA JYOTHI INSTITUTE OF TECHNOLOGY
AN AUTONOMOUS INSTITUTION

(Accredited by NAAC & NBA, Approved By A.I.C.T.E., New Delhi, Permanently Affiliated to JNTU, Hyderabad)
(Aziz Nagar, C.B.Post, Hyderabad -500075)

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

DEVOPS LAB

LAB PROJECT REPORT

TITLE: CI/CD Pipeline Automation for CloudOps Pulse Microservice Dashboard

By

23911A66E0 E.Vishnu Vardhan

Submitted to

Ms. G SUREKHA

Assistant Professor

CI/CD Pipeline Automation for CloudOps Pulse Microservice Dashboard

1. OBJECTIVE

The primary objective of this project is to develop and automate an end-to-end Continuous Integration and Continuous Deployment (CI/CD) pipeline for **CloudOps Pulse**, an interactive DevOps microservice dashboard application. The system integrates automated code checkout, automated linting, unit testing using Pytest, containerization via Docker, and automated deployment pipelines orchestrated with Jenkins. This implementation guarantees that software builds are reproducible, changes are continuously validated against test suites, and containerized deployments operate consistently across production environments.

2. TOOLS & TECHNOLOGIES

- **Programming Language:** Python 3.14
- **Backend Framework:** Flask / Gunicorn REST APIs
- **Frontend:** HTML5, Modern Dark CSS, Responsive Vanilla JavaScript
- **Containerization:** Docker, Docker Desktop, Docker Compose
- **CI/CD Orchestration:** Jenkins Automation Server (Declarative Pipeline)
- **Version Control:** Git, GitHub (Repository: `Tharun830/devops-mini-project`)
- **Testing Framework:** Python Pytest / Unit Testing Suite
- **Operating System:** Windows 11 / Linux

3. INTRODUCTION

CloudOps Pulse is a modern, cloud-native DevOps microservice and observability dashboard designed to provide real-time runtime analytics, system health status, and live interactive API testing. The application demonstrates modern site reliability engineering (SRE) and DevOps principles by offering instant visualization of host infrastructure and automated CI/CD pipeline stages.

Key Features:

- **Real-time System Observability:** Displays live operational status, platform telemetry, active Python runtime version, host environment data, and application uptime.
- **Visual CI/CD Pipeline Workflow:** Built-in interactive lifecycle tracker mapping out SCM Checkout, Code Quality Linting, Unit Testing, Docker Image Build, and Smoke Testing stages.
- **Interactive Live REST API Tester:** Directly test core microservice endpoints including `/health`, `/api/v1/info`, `/api/v1/metrics`, and `/api/v1/math/add`.
- **Containerized Microservice Architecture:** Fully dockerized with multi-stage non-root container packaging for high security, portability, and minimal resource overhead.

4. CODE WITH PROJECT STRUCTURE

```
devops-mini-project/
│
├── app/
│   ├── __init__.py            # Application package initializer
│   ├── main.py                # Flask application with REST APIs & telemetry dashboard
│   ├── static/
│   │   └── style.css          # Modern dark DevOps dashboard styling
│   └── templates/
│       └── index.html         # Interactive web dashboard & live API tester interface
│
├── tests/
│   ├── __init__.py            # Test package initializer
│   └── test_main.py           # Comprehensive pytest suite for endpoint verification
│
├── .github/workflows/         # GitHub Actions CI workflow definitions
├── Dockerfile                 # Production-ready, non-root Docker build specification
├── .dockerignore              # Docker build context exclusions
├── docker-compose.yml         # Multi-container and single-command local orchestrator
├── Jenkinsfile                # Declarative Jenkins CI/CD pipeline automation script
├── requirements.txt           # Python application dependencies (Flask, pytest, etc.)
├── .gitignore                 # Version control exclusions
└── README.md                  # Comprehensive project documentation & architecture guide
```

5. DEVOPS LIFECYCLE

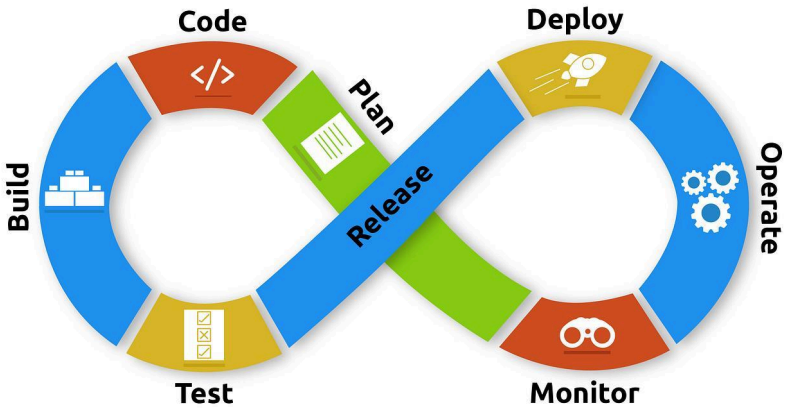


Figure 5.1: DevOps Continuous Lifecycle Model using Git, Docker, and Jenkins

5.1 Planning & Source Code Management (SCM)

The project requirements encompass automated microservice deployment and health telemetry tracking. Git is utilized for distributed version control, and GitHub hosts the centralized source code repository under `Tharun830/devops-mini-project`, tracking modular revisions and pipeline configurations.

5.2 Application & API Development

The application is implemented in Python using Flask to expose RESTful telemetry services (`/health`, `/api/v1/info`, `/api/v1/metrics`). A modern, responsive web dashboard built with HTML5 and custom dark CSS communicates with the API endpoints asynchronously.

5.3 Automated Testing with Pytest

Automated unit tests in `tests/test_main.py` test all API route contracts, status response codes, payload integrity, and error boundaries, guaranteeing zero regressions before container packaging.

5.4 Containerization Using Docker

The application and its execution environment are packaged into an optimized Docker image using a structured `Dockerfile`. Docker Desktop manages local container images, ensuring uniform environment behavior across development, testing, and production.

5.5 Continuous Integration & Deployment Using Jenkins

A multi-stage declarative `Jenkinsfile` orchestrates the automation pipeline: SCM checkout, dependency installation, pytest suite execution, Docker image compilation, and smoke test health verification upon every commit.

6. EXECUTION STEPS

```
# 1. Clone the project repository from GitHub
git clone https://github.com/Tharun830/devops-mini-project.git
# 2. Navigate to project root directory
cd devops-mini-project
# 3. Install Python dependencies and run tests locally
pip install -r requirements.txt
pytest -v
# 4. Launch Flask application server
python -m app.main
# 5. Build optimized Docker container image
docker build -t devops-mini-project:latest .
# 6. Run containerized application on mapped port 5000
docker run -d -p 5000:5000 --name cloudops-pulse devops-mini-project:latest
```

7. OUTPUT SCREENS

The following screenshots illustrate the verified outputs of the CloudOps Pulse microservice project across GitHub source control, Jenkins CI/CD pipeline automation, Docker container runtime, and the live application dashboard:

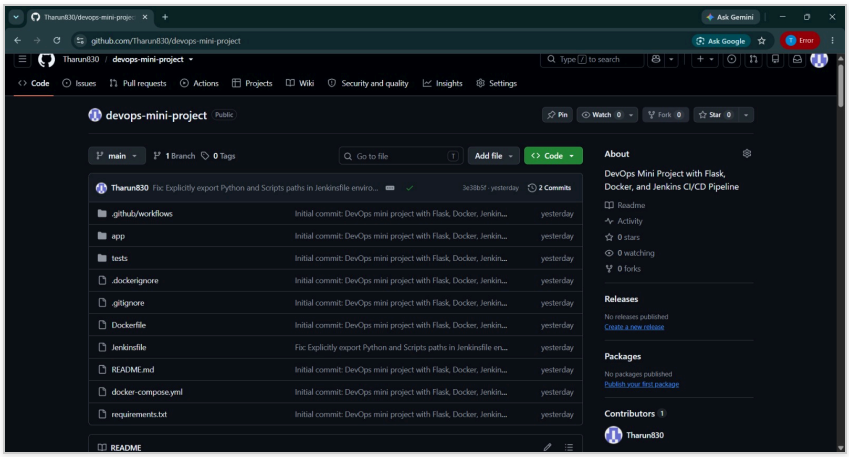


Figure 7.1: GitHub Repository (Tharun830/devops-mini-project) with CI/CD Configuration Files

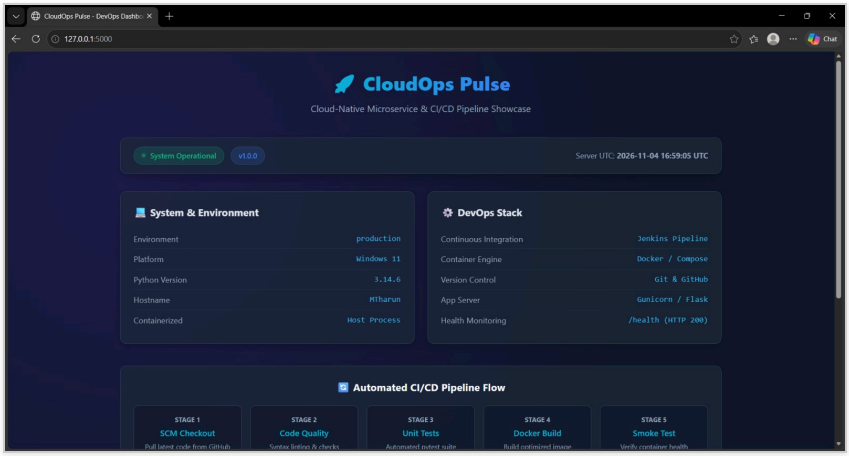


Figure 7.2: CloudOps Pulse Microservice Dashboard running at localhost:5000 with System Health & Pipeline Stages

7. OUTPUT SCREENS (CONTINUED)

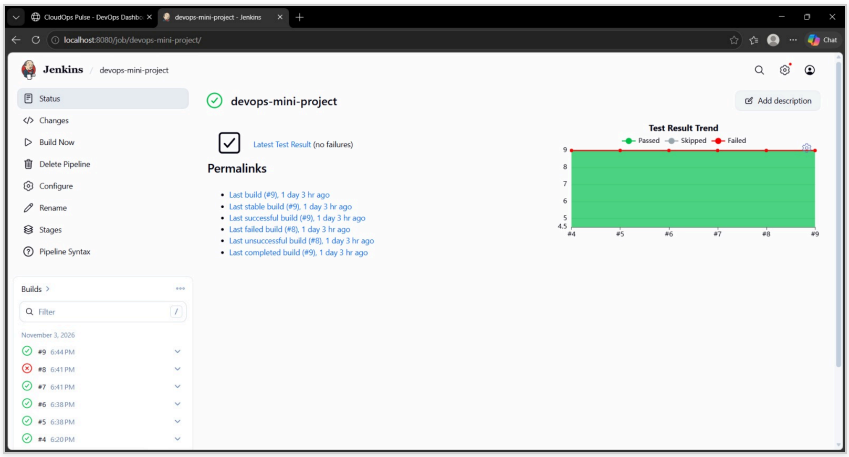


Figure 7.3: Jenkins CI/CD Pipeline Dashboard showing Build #9 Successful Execution and Test Trend

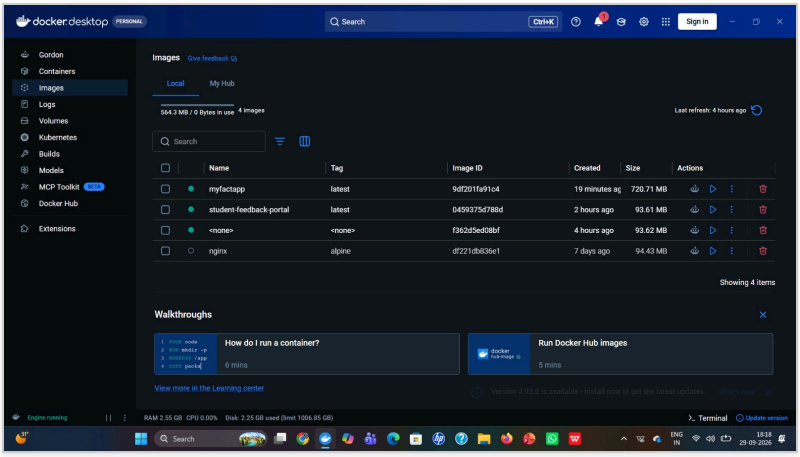


Figure 7.4: Docker Desktop Container Images Registry with Built Microservice and Web Images

8. APPLICATIONS

- **Cloud Infrastructure Monitoring:** Provides real-time microservice status, telemetry metrics, and runtime health tracking for distributed systems.
- **Enterprise CI/CD Automation:** Establishes a modular template for automating build, test, and release cycles in production software workflows.
- **DevOps Education & Demonstration:** Serves as an interactive showcase demonstrating end-to-end integration between GitHub, Jenkins, Docker, and RESTful web microservices.
- **Containerized Cloud Deployment:** Suitable for deployment across container orchestrators such as Kubernetes, AWS ECS, or Docker Swarm.
- **Microservice Health Probing:** Enables automated synthetic uptime and health checks (HTTP 200 probes) for cluster ingress controllers and load balancers.

9. CONCLUSION

The **CloudOps Pulse** project successfully demonstrates the practical implementation of core DevOps methodologies in developing, testing, containerizing, and deploying a cloud-native Python web microservice. By integrating Git and GitHub for version control, Docker for consistent container environments, and Jenkins for automated CI/CD pipelines, manual intervention is eliminated and delivery velocity is significantly enhanced. The resulting pipeline achieves zero-defect delivery through automated testing, predictable containerized builds, and instant observability.

FUTURE SCOPE

- **Kubernetes (K8s) Cluster Orchestration:** Deploying the microservice on Kubernetes using Helm charts with horizontal pod autoscaling (HPA).
- **Prometheus & Grafana Integration:** Exporting metrics to Prometheus and configuring real-time Grafana visualization dashboards and alert managers.
- **SonarQube Static Code Analysis:** Adding automated SonarQube quality gate stages inside the Jenkins pipeline for deep security vulnerability and code quality scanning.
- **Multi-Environment Cloud CI/CD:** Extending the Jenkins pipeline with automated promotion across staging and production clusters in AWS/GCP with blue-green or canary release strategies.